💡 1️⃣ What is System Design?

System Design means **planning how to build a large-scale software system** so that it is:

- **Scalable** (handles more load)
- **Reliable** (keeps working even if some parts fail)
- **Maintainable** (easy to add features or fix issues)
- **Efficient** (uses resources wisely)

In short:

🧠 *It's how we design the blueprint of a big system (like Netflix, Uber, or Instagram) to handle millions of users.*

⚙️ 2️⃣ Why Scaling Matters

When your app grows:

- More **users**
- More **data**
- More **requests per second**

Your single server can't handle all of it.
That's when **scaling** becomes necessary.

Example:
A small blog on one server is fine.
But if 10 million users visit at once → you need multiple servers, caching, and load balancing.

🧩 3️⃣ Types of Scaling

🚀 (a) Vertical Scaling (Scale Up)

Idea: Increase power of one machine.
E.g., upgrade CPU, RAM, or SSD.

✅ **Pros:**

- Simple to implement (no code change)
- Works well for moderate traffic

❌ **Cons:**

- Expensive hardware
- Physical limits (you can't keep adding CPU forever)
- Single point of failure

Use when: System is small or just starting out.

⚡ **(b) Horizontal Scaling (Scale Out)**

Idea: Add more machines (servers) and distribute load.
E.g., instead of 1 powerful server → use 10 average servers.

✅ **Pros:**

- Practically unlimited growth
- Redundancy (if one fails, others work)

❌ **Cons:**

- Requires **load balancers**
- More complex (data consistency, deployment)

Use when: You need **high scalability** (like Google, Amazon, Netflix).

🧱 **(c) Sharding (Data Partitioning)**

Idea: Split your data into smaller chunks called **shards** and store them across multiple databases.

Example:
If you have users from multiple regions:

- Shard 1 → users from Asia

- Shard 2 → users from Europe
- Shard 3 → users from US

✅ **Pros:**

- Each DB handles less load
- Enables horizontal scaling for databases

❌ **Cons:**

- More complex queries
- Harder to rebalance shards later

Use when: Database becomes too large or too slow.

⚡ **(d) Caching**

Idea: Store **frequently used data in memory** (fast access) instead of fetching from DB every time.

Example:

- User profile or homepage data stored in Redis or Memcached.

✅ **Pros:**

- Much faster responses (microseconds)
- Reduces database load

❌ **Cons:**

- Stale data (cache may not update immediately)
- Memory cost (RAM is expensive)

Use when: Data is read often and doesn't change frequently.

🧱 **Common Caching Layers**

Cache Type	Example	Stored Where	Use Case
Client-side	Browser cache	User's browser	Static files (images, CSS, JS)
CDN (Edge)	Cloudflare, Akamai	Edge servers	Images, videos, static content
Application cache	Redis, Memcached	App layer	User sessions, precomputed results
Database cache	Query cache	DB layer	Common query results

🧠 ⚡ CAP Theorem

CAP = Consistency, Availability, Partition Tolerance

It states:

In a distributed system, you can only guarantee **2 out of 3**.

Property	Meaning
Consistency (C)	Every read gets the latest write
Availability (A)	Every request gets a response (even if outdated)
Partition Tolerance (P)	System works even if network breaks between servers

---

🧩 CAP Trade-offs

System Type	Chosen Two	Example
CP (Consistency + Partition tolerance)	Sacrifice Availability	HBase, MongoDB (strict mode)
AP (Availability + Partition tolerance)	Sacrifice Consistency	Cassandra, DynamoDB

System Type	Chosen Two	Example
CA (Consistency + Availability)	No partition tolerance (single server)	Relational DBs (MySQL on one node)

🧠 Easy Example:

Imagine two servers storing balance = ₹1000.
If one user updates it to ₹900, and network between servers breaks:

- **Consistency focus (CP)** → second server won't respond until sync happens.
- **Availability focus (AP)** → second server will respond but may show old ₹1000.

Both are valid, depends on business need.

🧩 5️⃣ Real-life Example

System	Chosen Strategy
Banking	Consistency (you can't afford incorrect balance)
Social Media Likes	Availability (show likes instantly, sync later)
Search Engines	Availability + Partition tolerance
E-commerce stock count	Balance between both (eventual consistency)

🧠 Key Idea

Distributed systems are a trade-off between **speed, correctness, and fault tolerance**.

🧩 6️⃣ Summary Table

Concept	What it means	When to use
Vertical Scaling	Bigger machine	Simple but limited

Concept	What it means	When to use
Horizontal Scaling	More machines	For high traffic
Sharding	Split DB by data	For big databases
Caching	Store hot data in memory	For fast reads
CAP Theorem	Choose 2 out of 3 (C, A, P)	Depends on use case

🚀 **Interview Tip**

If interviewer asks:

“How would you scale your system if traffic suddenly grows?”

Say:

“First, I’d add caching (Redis/CDN) to reduce DB load, then move toward horizontal scaling with load balancers and data sharding.”

Nice — let’s design a **practical, interview-friendly** deep dive on **Load Balancers** with the key concepts and a small design walkthrough you can present clearly. I’ll keep it simple and usable in an interview or take-home.

Load Balancer — simple explanation

A **Load Balancer (LB)** is a network component that **distributes incoming client requests across multiple backend servers** to improve throughput, reliability and availability. It sits between clients and your server fleet and decides **which server** should handle each request.

Primary goals:

- Evenly distribute traffic
- Protect backend from overload
- Provide high availability (failover)
- Improve latency by routing to healthy/closest servers

Core concepts (short & clear)

L4 vs L7

- **L4 (Transport layer)**: routes based on IP/port (TCP/UDP). Fast, simple (e.g., TCP load balancing).
- **L7 (Application layer)**: routes based on HTTP fields — host, path, headers, cookies. Smarter routing (e.g., path-based routing, A/B).

Reverse Proxy

A **reverse proxy** is how many LBs operate at L7: LB receives requests and *proxies* them to backend servers. Clients never see backends directly.

Load balancing algorithms

- **Round Robin** — rotate through servers.
- **Weighted Round Robin** — more powerful servers get more traffic.
- **Least Connections** — pick server with fewest active connections.
- **IP Hash / Consistent Hashing** — route requests by client IP (or hashed key) — useful for affinity.
- **Least Response Time, Random, Resource-based** — specialized choices.

Sticky Sessions (Session Affinity)

- **Definition:** route requests from the same client to the *same* backend instance.
- **Methods:** cookies (inserted by LB), IP hash, application cookie.
- **Why use:** legacy apps store session in memory on an instance.
- **Downside:** reduces load distribution flexibility; causes hotspotting; breaks when instance dies.
- **Better alternative:** make services *stateless* and store session in a shared store (Redis).

Health Checks

- LB periodically pings backends to detect healthy/unhealthy instances.

- **Types:** TCP probe (is port open), HTTP probe (GET /health), custom deep check (DB connectivity).
- **Actions:** stop routing to unhealthy nodes; when recovered, put back in rotation.
- Use **active** (periodic) and **passive** (on failure observed) checks together.

SSL/TLS termination

- **Terminate at LB** (offload SSL): LB decrypts TLS, forwards plain HTTP to backends — reduces CPU on app servers and allows L7 routing.
- **End-to-end encryption:** LB passes through TLS (or re-encrypts to backend) if required for security.

Connection draining (graceful shutdown)

When removing a backend (maintenance), LB should stop sending new requests but finish in-flight requests before shutdown.

Sticky & Stateful trade-offs

- Sticky sessions are quick fix; long-term solution: stateless services + shared session store or JWT.

Design Mini — "Design a Load Balancer"

I'll present this as: Requirements → Constraints → High-level design → Components & flow → Important details & interview talking points.

Requirements (simple)

Functional

- Accept client HTTP(S) requests and distribute to many web servers.
- Support path-based routing (e.g., /api → api pool, /static → CDN or static pool).
- Health check of backends; automatic failover.
- TLS (HTTPS) support.
- Optionally support session affinity.

Non-functional

- Handle tens of thousands → millions RPS (scale horizontally).

- Low latency (small added overhead).
- Highly available (no single point of failure).
- Observability: metrics, logs.

Constraints & trade-offs

- If you **terminate TLS** at LB: CPU on LB, but easier routing and monitoring.
- If you need strict **consistency/affinity** (stateful sessions), you lose some load distribution and scaling benefits.
- Choosing **L4** gives speed; **L7** gives flexibility.

High-level architecture (text diagram)

Clients

|

v

DNS (round-robin or geo-DNS)

|

v

Edge / CDN (for static files, caching)

|

v

Load Balancer cluster (L4/L7) <-- auto-scale group

| | |

v v v

App1 App2 AppN

\ | /

-> Redis (session/cache) (optional)

-> Database (primary + replicas)

Components & responsibilities

1. **DNS** — maps hostname to LB IP(s). Use multiple LB IPs (or anycast).
2. **Edge / CDN** — caches static content and reduces load on LBs/backends.
3. **Load Balancer cluster**
    - Accepts TLS (optionally)
    - Performs routing (L4 route / L7 path/host-based)
    - Performs health checks
    - Implements LB algorithm
    - Tracks active connections, metrics
    - Offers sticky session feature or sets cookie
4. **Service pool (App servers)** — stateless application instances
5. **Shared session store** (Redis) — if you want sessions without sticky
6. **Data stores** — DB, caches, etc.
7. **Autoscaler & Orchestration** — spawn/terminate app instances, update LB pool
8. **Monitoring & Alerting** — LB metrics (latency, error rate, active connections), backend metrics

Request flow (simple)

1. Client → DNS gives LB IP.
2. Client opens TLS to LB (or LB forwards TCP).
3. LB receives request → runs routing rules (path/host).
4. LB does health check on candidate backend(s).
5. LB picks backend (algorithm); forwards request.
6. Backend responds → LB sends response back to client.
7. LB updates metrics; if backend error, can perform passive health marking.

Session affinity options (recommendation)

- **Preferred:** Make app stateless + store session in Redis / JWT in client.

- **If you must use sticky:** use application cookie set by LB (or server) or use consistent hashing (e.g., hash(session_id) → shard backend). For resilience, use consistent hashing so when backends change, small fraction of keys move.

Failure handling

- If a backend fails health checks → LB stops sending new traffic and optionally drains in-flight.
- If LB instance fails → DNS/anycast + LB cluster ensures HA.
- Autoscale: if CPU high / latency up → spawn more backends and register with LB.

Implementation checklist (practical steps / features to mention in interview)

- Use **two or more LB nodes** behind DNS for HA.
- Implement **health checks** (HTTP /health returning 200); remove unhealthy nodes.
- **Graceful shutdown**: drain connections before shutdown.
- **Metrics**: request rate, latency percentiles, 5xx rate, backend health.
- **Logging**: access logs, error logs.
- **Rate limiting / circuit breaker** at LB for abusive traffic.
- **SSL offload** at LB (or passthrough if needed).
- **Autoscaling** backend instances + dynamic LB reconfiguration (service discovery).
- **Blue-green / canary deploy** via LB routing rules (e.g., route 5% to new version).

Short config examples (very short & interview-ready)

Nginx (simple sticky by ip_hash, basic)

```
upstream backend {
  ip_hash;          # sticky by client IP
  server 10.0.0.1:80;
  server 10.0.0.2:80;
}
```

```
server {
  listen 443 ssl;
  ssl_certificate ...;
  location / {
    proxy_pass http://backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
  }
}
```

HAProxy (health checks, round-robin)

```
frontend http-in
  bind *:80
  default_backend servers

backend servers
  balance roundrobin
  server s1 10.0.0.1:80 check
  server s2 10.0.0.2:80 check
```

(Use these snippets to quickly show you know how to configure common LBs.)

Interview talking points — short answers you can rehearse

- **Q:** Where do you terminate TLS?
  A: Usually at the LB for easier routing and lower CPU on app servers; for extra security, re-encrypt to backends.
- **Q:** How do you implement session affinity?
  A: Prefer stateless apps + shared session store (Redis). If must, LB cookie or consistent hashing.

- **Q:** How to scale the LB itself?
  A: Run multiple LB instances behind DNS/anycast, use autoscaling, and make them stateless so they can be replaced.
- **Q:** How do you avoid sending traffic to unhealthy instances?
  A: Active health checks + passive failure detection; use connection draining when removing nodes.
- **Q:** How do you route by path?
  A: Use L7 LB (reverse proxy) to route based on host/path rules.

Short summary (one-paragraph)

A Load Balancer distributes traffic across servers to make systems scalable and available. Choose L4 when you need raw speed and L7 when you need flexible routing (path/host/cookie). Prefer stateless backends and shared session stores over sticky sessions, and always use health checks, connection draining, autoscaling integration, and observability. For HA, run multiple LB instances and use DNS/anycast.

Want a one-page printable **diagram + checklist PNG** for this load balancer note (so you can carry it into interviews)? I can create that next.

⚙️ 1️⃣ Caching Systems — Basics

💾 What is a Cache?

A **cache** is a *fast, in-memory* data store used to temporarily hold frequently accessed data so future requests are faster.

Example:
Instead of hitting the database every time to fetch a user profile → store it in Redis.

Goal: Reduce latency, reduce DB load, and improve user experience.

⚡ 2️⃣ Redis Basics

Redis (Remote Dictionary Server) is an **in-memory key-value store**, often used as:

- Cache
- Session store
- Message broker (via Pub/Sub)
- Rate limiter
- Leaderboard (sorted sets)

Why Redis?

- Extremely fast (microseconds latency)
- Supports persistence (RDB, AOF)
- Data structures: String, List, Set, Sorted Set, Hash
- Built-in eviction, TTL, replication, and clustering

Protocol:
Redis uses **RESP (REdis Serialization Protocol)** over **TCP**.

- Lightweight binary-safe protocol
- Commands like SET key value, GET key

Typical Setup:

- App → Redis cache → Database (like Postgres/MySQL)

🧱 3️⃣ Cache Eviction Policies

When Redis runs out of memory, it needs to remove old entries.
This is handled by **Eviction Policies** (set via maxmemory-policy).

Policy	Meaning	Example Use
noeviction	Error on new write when full	Financial or critical data
volatile-lru	Evict least recently used among keys with TTL	Session data
allkeys-lru	Evict least recently used key overall	General caching
volatile-lfu	Evict least frequently used (with TTL)	API cache

Policy	Meaning	Example Use
allkeys-lfu	Evict least frequently used key overall	Smart cache eviction

---

🔁 🔼 **Cache Patterns**

🧩 **Cache-Aside (Lazy Loading) — Most Common**

Flow:

1. App first checks **cache**.
2. If key **found (cache hit)** → return data.
3. If **miss** → fetch from DB, store in cache, return.

App → Redis → DB

Pros:

- Simple and flexible
- DB always source of truth

Cons:

- Cache miss causes extra latency
- Potential for stale data

✅ **Best For:** Read-heavy systems (e.g., product catalog, user profiles)

🖍️ **Example:**

```
value = redis.get("user:123")
if not value:
  value = db.query("SELECT * FROM users WHERE id=123")
  redis.set("user:123", value, ex=3600)
return value
```

📐 Design Mini: URL Shortener

💡 Problem Statement

Design a **URL shortener** like bit.ly:

- Input: Long URL
- Output: Short unique URL (e.g., https://short.ly/XyZ123)
- Redirect short URL → original long URL
- Must scale to millions of URLs
- Handle high read traffic (redirects)

🎯 Functional Requirements

- Shorten a given URL (POST /shorten)
- Redirect when user hits short URL (GET /{code})
- Handle duplicates (same URL = same code)
- Optional: URL expiry (TTL)
- Analytics: track clicks

⚙️ Non-Functional Requirements

- Low latency (sub-50ms)
- Highly available
- Scalable (reads >> writes)
- Reliable (no duplicate short codes)
- Cost-effective storage

🌐 Step 1 — API Design

Operation	Method	Endpoint	Description
Shorten URL	POST	/shorten	Takes long URL, returns short one
Redirect	GET	/{code}	Redirects to long URL
Analytics	GET	/stats/{code}	Optional

Protocol:

- Use **HTTP REST** (simple, widely supported).
- For internal service communication (e.g., between cache and DB), **TCP**.

💾 Step 2 — Database Design

We need:

1. **Mapping Table:** short_code ↔ long_url
2. **Optional:** click_count, created_at, TTL

Schema Example:

short_code (PK)	long_url	created_at	click_count	expiry
XyZ123	https://google.com	2025-10-31	100	NULL

Choice:

🧱 **Primary DB: NoSQL (e.g., DynamoDB / Cassandra / MongoDB)**

- Handles large scale, high read/write
- No joins
- Easy horizontal scaling

🟡 **Alternative: PostgreSQL/MySQL** (if small scale or ACID required)

⚡ Step 3 — Short Code Generation

Approaches:

1. **Base62 Encoding**

- Take an integer ID (auto-increment or distributed counter)
- Convert to Base62 (a–z, A–Z, 0–9)
- Example: 125 → “cb”

2. **Hashing (MD5/SHA)**
   - Hash long URL, take first 6–8 chars
   - Handle collisions with rehashing
3. **UUID-based (less readable)**
   - Use for distributed generation, but longer URLs

✅ **Recommendation:**
Use **Base62 encoding** with distributed counter (Redis INCR or DB sequence).

🚀 **Step 4 — Caching Layer (Redis)**

Because **read traffic is 100x write traffic**,
Redis is used to cache:

Key	Value	TTL
short:XyZ123	https://google.com	24 hrs

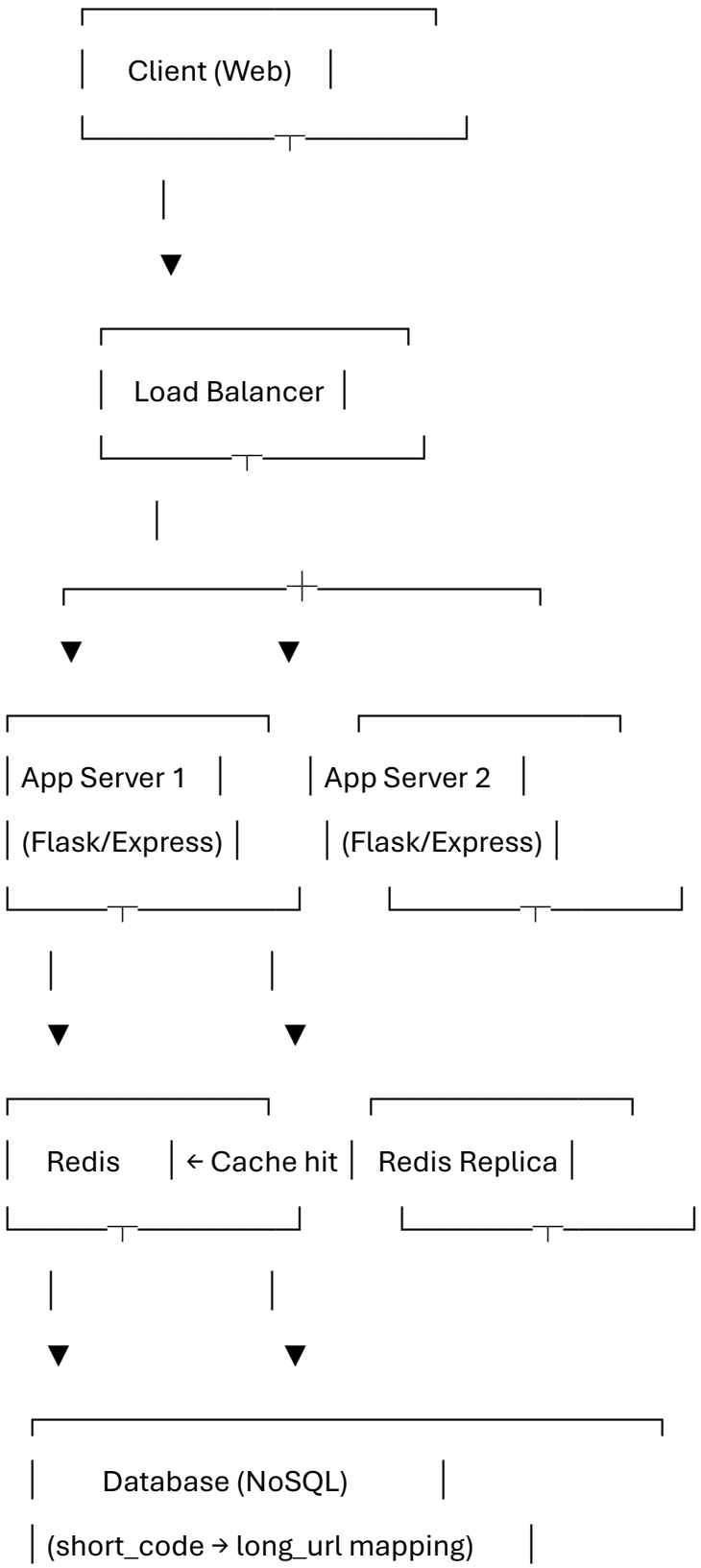
Flow:

1. GET /XyZ123
2. App checks Redis:
   - ✅ If hit → redirect
   - ❌ If miss → query DB, set in Redis

Why Redis?

- Microsecond latency
- Handles millions of reads/sec
- TTL for automatic cleanup

🧭 Step 5 — High-Level Architecture Diagram



🔐 Step 6 — Protocols & Justification

Component	Protocol	Why
Client → App	**HTTP/HTTPS**	REST-based API, browser friendly
App → Redis	**RESP (TCP)**	Redis uses binary-safe TCP protocol
App → DB	**TCP (JDBC/Driver)**	Standard DB connection
Load Balancer	**HTTP(S) / TCP**	Routing traffic between servers

---

🧠 Step 7 — Scaling Strategy

Component	Scaling	Tool
App Layer	Horizontal	Auto-scaling groups
Cache	Redis Cluster	Sharding for large datasets
DB	Sharding + Read Replicas	DynamoDB partitioning
LB	Multiple nodes + health checks	AWS ELB / Nginx
Code Generation	Distributed counter	Redis INCR / Snowflake ID

Bottlenecks:

- DB writes → Use async write queue
- Cache invalidation → Set TTL, or lazy delete

🧮 Step 8 — Cost Calculation (Example for Interview)

Let's estimate for **100M URLs** and **1B requests/day** (mostly reads):

Component	Estimate	Cost (AWS Example)
Redis (cache)	5GB memory, 10k RPS	~$60/month (Elasticache small node)
DynamoDB	100M rows	~$200/month
EC2 App servers (3x t3.medium)	Handle 1000 RPS each	~$120/month
Load Balancer	AWS ALB	~$20/month
Total		**~$400/month**

🧠 *Always mention: cost can be optimized with CDN caching for static redirect pages.*

🧩 Step 9 — Key Interview Talking Points

Question	Example Answer
Why Redis?	In-memory store, microsecond latency, built-in TTL.
Why NoSQL?	Simpler key-value lookup, horizontally scalable.
How to avoid duplicate short codes?	Unique index on short_code or centralized counter.
How to handle cache invalidation?	TTL or lazy update on write.
What if Redis crashes?	Rebuild cache from DB (source of truth).
How to scale for millions of requests?	Use CDN + Redis + multiple app servers behind LB.
How to ensure fault tolerance?	Redis replication + DB replicas + multiple LBs.
How to prevent abuse (rate limiting)?	Redis-based rate limiter using INCR with TTL.

📜 Step 10 — TL;DR Summary

Layer	Technology	Purpose
API	REST over HTTP	Shorten + redirect endpoints
Load Balancer	Nginx / AWS ELB	Distribute requests
Application	Flask / Node / Spring Boot	Core business logic
Cache	Redis	Fast read + TTL expiry
Database	DynamoDB / PostgreSQL	Persistent storage
Monitoring	CloudWatch / Prometheus	Metrics + alerts

🎨 Bonus: Elevator Pitch (30-sec Interview Summary)

"I'd design the URL Shortener as a stateless HTTP service behind a load balancer. Each new URL is stored in a NoSQL DB with a short code generated using Base62 encoding of a distributed counter. I'd use Redis as a cache-aside layer for fast lookups on redirects and TTL for expired links. The system scales horizontally across app servers, Redis shards, and DB partitions. All communication is over HTTP and TCP protocols. Estimated cost for 100M URLs and 1B daily requests is about $400/month using AWS."

Perfect — let's go **step by step** and make this your **System Design Day 3 Master Sheet** (SQL/NoSQL + Key-Value Store design + costing).
I'll keep it interview-friendly, simple, and *ready to speak out loud* with logical flow and example numbers.

🧩 1️⃣ SQL vs NoSQL – Quick Summary

Feature	SQL (Relational)	NoSQL (Non-Relational)
Data model	Tables (rows & columns)	Key-Value, Document, Column, Graph
Schema	Fixed (predefined schema)	Flexible (schema-less)
Scaling	Vertical (scale-up)	Horizontal (scale-out)

Feature	SQL (Relational)	NoSQL (Non-Relational)
Query	Powerful JOINs, ACID	Simpler CRUD (no JOINs)
Use Case	Financial data, transactions	Big data, caching, logs, IoT
Examples	MySQL, PostgreSQL	MongoDB, Cassandra, Redis, DynamoDB

✅ **Interview Tip:**
👉 If your data has *relationships* and *transactions* → use SQL.
👉 If you need *scalability* and *speed for unstructured data* → use NoSQL.

🧩 2️⃣ Indexing, Sharding, Replication – Core Concepts

🔹 Indexing

- Like a **book index** – helps find rows faster.
- Uses **B-Tree or Hash** structures.
- Speeds up SELECT but slows INSERT/UPDATE (because index must update too).

✅ **When to use:**
When you frequently query specific columns (WHERE email = ?, ORDER BY date).

🔹 Sharding

Splitting large dataset into **smaller logical chunks (shards)**.

Type	Example	Notes
Range-based	Users A–M in Shard1, N–Z in Shard2	Simple, but can create hotspots
Hash-based	userId % 4 → shard	Even load, harder to query across shards
Geo-based	Region = "APAC" → Shard3	Good for global systems

✅ Helps **horizontal scaling**
❌ Increases **complexity in queries & joins**

🔹 **Replication**

- Keeping **copies of same data** across servers for **fault tolerance**.
- **Leader–Follower model:**
    - Writes → go to Leader
    - Reads → can go to Followers
- Used for **high availability** and **disaster recovery**.

✅ Reduces downtime
❌ Risk of data inconsistency (eventual consistency)

🧩 3️⃣ **ACID vs BASE**

Concept	ACID (SQL)	BASE (NoSQL)
Atomicity	All or nothing	—
Consistency	Always valid	Eventually consistent
Isolation	No conflicts	—
Durability	Safe even if crash	Best effort persistence
Use Case	Banking, transactions	Social feed, caching, logs

✅ **Interview tip:**
"ACID ensures correctness; BASE ensures scalability."

💡 **DESIGN MINI: "Design a Key-Value Store (like Redis)"**

1️⃣ **Functional Requirements**

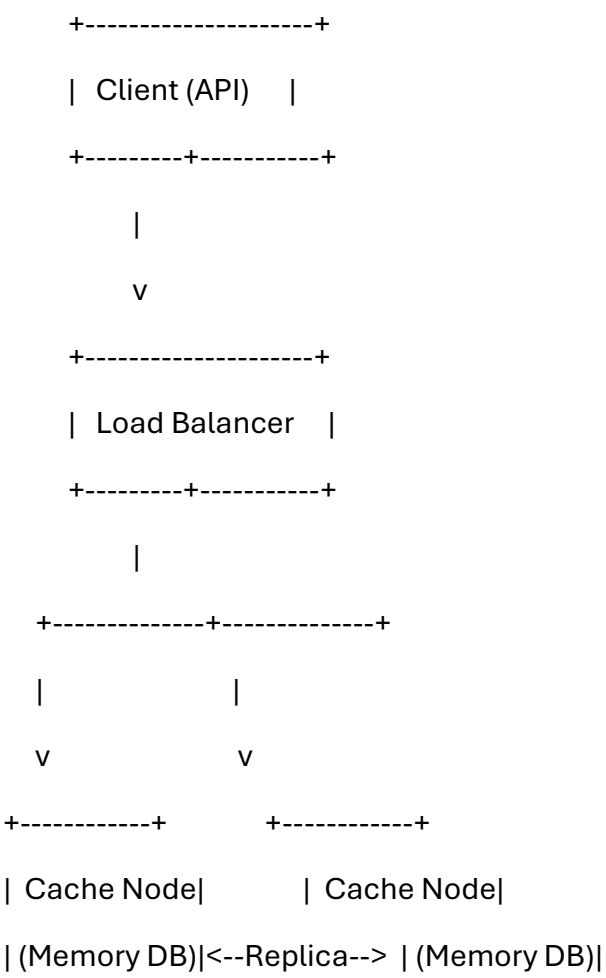
- SET key value
- GET key

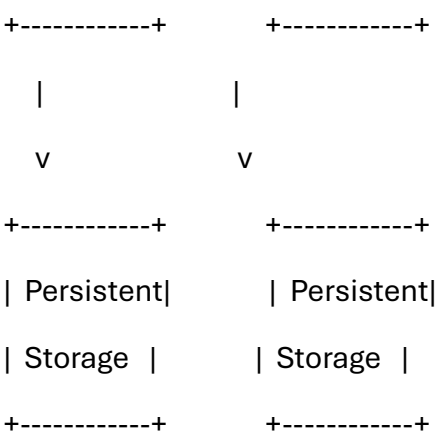
- Optional: TTL (expire keys)
- Handle millions of reads/writes per second
- Must be fast (low latency < 1ms)

2️⃣ Non-Functional Requirements

- High availability
- In-memory for speed
- Scalable horizontally
- Durable (optional backup to disk)

3️⃣ High-Level Architecture





4️⃣ Components Explained

Component	Role
Client API	Provides GET/SET interface
Load Balancer	Distributes traffic across cache nodes
Cache Node	In-memory store (e.g., using hash tables)
Replication	Keeps backup for fault tolerance
Persistence Layer	Optional backup (RDB, AOF like Redis)

5️⃣ Protocols & Choices

Feature	Choice	Why
Communication	TCP or RESP protocol	Low-latency binary/text
Storage	In-memory hash map	O(1) access time
Persistence	Append-only file (AOF)	Crash recovery
Replication	Master–Replica	High availability
Sharding	Consistent Hashing	Smooth scaling without reshuffling

---

6️⃣ Example Memory & Cost Calculation

Let's assume:

- Each key = 20 bytes
- Each value = 100 bytes
- Redis overhead per entry ≈ 50 bytes
- Total per record ≈ **170 bytes**

If we have **10 million keys**:

10,000,000 × 170 bytes = 1.7 GB total

To maintain headroom (for rehashing, OS memory, and replication):

👉 **Use 2x headroom:** ~3.4 GB
👉 With replication (1 replica): ~6.8 GB

✅ **So 1 node with 8 GB RAM** can handle ≈ 10M keys safely.

7️⃣ Cost Estimation (Example on AWS)

Resource	Example Service Cost (per month)
EC2 t3.large (8 GB RAM)	$50
Replication Node	$50
Storage (EBS backup 10 GB)	$1
Total (per region)	**≈ $100/month**

💡 *To scale horizontally:*
Add more nodes using consistent hashing (each node handles its key range).

8️⃣ Key Optimization Strategies

- Use **LRU eviction** to manage full memory.
- Add **write-ahead logs (AOF)** for durability.

- **Batch writes** or **pipeline commands** for throughput.
- **Monitoring:** Memory usage, hit/miss ratio, latency.

🚦 Common Interview Flow

If interviewer says “design Redis”:

1. Start with **requirements** (read-heavy, in-memory, low-latency)
2. Sketch high-level architecture
3. Talk **storage design (hash table)**
4. Mention **replication + sharding**
5. Show **scaling and fault tolerance**
6. Estimate **memory & cost**
7. End with **trade-offs** (e.g., memory vs persistence)

Perfect 👌
Let’s make this your **System Design – Day 4: Messaging & Notification Systems (Cheat Sheet Edition)**

This is one of the most **commonly asked interview designs** — especially in distributed systems or backend interviews (e.g. “Design a Notification Service like Facebook/Slack/WhatsApp”).

I’ll explain it in simple language + structured flow + include cost/memory/infra assumptions + a high-level diagram.

🧩 1️⃣ Kafka, RabbitMQ, Pub/Sub — Basics

Feature	**Kafka**	**RabbitMQ**	**Pub/Sub Concept**
Type	Distributed commit log	Message broker (queue-based)	Pattern for async communication
Use Case	High-throughput event streaming	Reliable message delivery	Broadcast-style

communication |
Model	Pull (consumer fetches messages)	Push (broker sends to consumers)	One publisher → many subscribers
Ordering	Partition-based ordering	Queue-level ordering	Usually best-effort
Persistence	Messages stored on disk	Optional (in-memory or disk)	Depends on implementation
Example Use	Logging, metrics, pipelines	Task queues, notifications	Fan-out updates

✅ **In short:**

- Use **Kafka** when you want scalable, high-throughput, event logs.
- Use **RabbitMQ** when reliability and routing are critical.
- **Pub/Sub** is the **pattern** behind both (Publisher → Broker → Subscribers).

🧩 2️⃣ Producer–Consumer Model (Core Concept)

Think of it like a post office system:

- **Producers** send messages (emails).
- **Broker** stores messages temporarily.
- **Consumers** pick them up asynchronously.

🎯 This model **decouples systems** → producer and consumer don't need to know each other.

Why it matters:
✅ Handles traffic spikes
✅ Improves reliability
✅ Enables async processing

🧩 3️⃣ Offset Management (in Kafka-style systems)

- Every message in Kafka has an **offset** = unique ID in a partition.
- Consumers **commit** offsets after processing (so they can resume later).
- **Consumer group** ensures that each message is processed once per group.

🧠 Example:

Partition 0: [0][1][2][3][4]

Consumer1 offset = 3 → processed till message 3

If Consumer1 crashes, another consumer can start from offset 3.

💡 **DESIGN MINI: "Design a Notification System"**

1️⃣ **Problem Statement**

Design a system to send notifications (Email/SMS/Push) for events like:

- Order placed
- Payment success
- Comment received

It must handle **millions of notifications/day**, be **reliable**, and **scalable**.

2️⃣ **Requirements**

Functional:

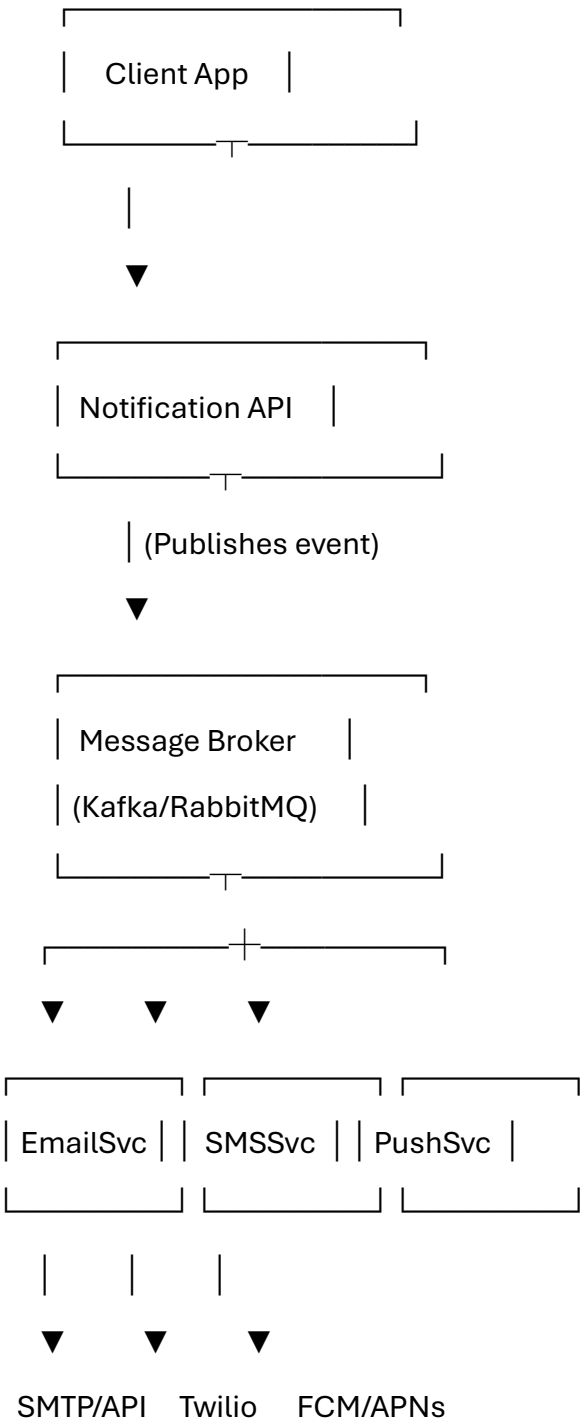
- APIs to create notification requests
- Support multiple channels (Email, SMS, Push)
- Status tracking (Success/Failure)

Non-Functional:

- High availability
- Asynchronous delivery
- Retry mechanism for failed notifications
- Handle spikes (e.g., festival sales)

---

3 High-Level Architecture



---

4 Component Explanation

Component	Responsibility
Notification API	Entry point, receives request, validates input
Message Broker (Kafka/RabbitMQ)	Stores events temporarily, decouples producers & consumers
Email/SMS/Push Workers	Consume messages, call external APIs
External Services	Send actual notification (e.g., Gmail SMTP, Twilio, Firebase)
Database (PostgreSQL/Redis)	Store notification logs, status, retries
Retry Queue (Dead Letter Queue)	Handles failed messages and retries later

---

5️⃣ Data Flow

1️⃣ Client triggers notification → POST /notify
2️⃣ API stores request in DB + pushes to **Kafka topic** (e.g., notifications)
3️⃣ Consumers (workers) read from topic by channel type
4️⃣ Each consumer sends notification (via SMTP/Twilio/FCM)
5️⃣ On success/failure → update DB
6️⃣ Failed messages → DLQ for retry

6️⃣ Protocols Used

Layer	Protocol	Why
API	HTTPS	Secure client communication
Broker	TCP (Kafka or AMQP for RabbitMQ)	Reliable message delivery
External APIs	SMTP / HTTPS	Channel-specific communication

7️⃣ Scaling Strategy

Component	Scaling Approach
API Layer	Behind Load Balancer (Nginx/ELB)
Message Broker	Partitioned Topics (Kafka)
Workers	Auto-scale horizontally
DB	Sharding by user_id, replication for reads
Caching	Redis for message deduplication

8️⃣ Cost & Memory Estimation Example

Let's assume:

- 10M notifications/day (~115 notifications/sec)
- Each message = 1 KB

Resource	Example	Estimation
Message Data/Day	10M × 1KB	~10 GB/day
Kafka Retention (7 days)	10 GB × 7	70 GB storage
Kafka Cluster (3 brokers)	3 × m5.large	~$300/month
Worker Nodes (autoscaling)	5 × t3.medium	~$150/month
DB (RDS PostgreSQL)	db.t3.medium	~$60/month
Total Infra Estimate		**≈ $500–$600/month**

9️⃣ Reliability & Optimization

✅ **Retry Policy:**
Exponential backoff + DLQ for failed deliveries.

✅ **De-duplication:**
Use Redis to avoid duplicate messages if same event retried.

✅ **Monitoring:**

- Message lag in Kafka
- Failed notification rate
- API latency

✅ **Security:**

- HTTPS + API Key Auth
- Encrypt user data

🔟 **Interview-Ready Summary (How to Speak It)**

"I'll design a distributed Notification System using a Pub/Sub model.
The Notification API publishes messages to Kafka. Each channel-specific worker consumes from its partition and pushes notifications via third-party APIs (SMTP, Twilio, FCM).
Kafka ensures durability and async delivery; DLQs handle failures.
The system scales horizontally with autoscaling workers and multiple partitions.
On AWS, this setup with 10M notifications/day would cost around $500–600/month.
We can further optimize cost by compressing messages, batching sends, and using Redis caching."

Nice — let's design this clearly and interview-ready. I'll keep language simple, show the core algorithms (leader election, replication, consistency), then walk through a concrete **Distributed Lock Service** design (like ZooKeeper / etcd), with architecture, APIs, failure-handling, trade-offs, and a small cost/memory estimate you can speak to in an interview.

🎯 **High-level goal (one sentence)**

A **distributed lock service** provides **mutual exclusion** across processes/nodes in a distributed system: clients can acquire a lock on a key, do work, and release the lock — with guarantees about correctness even when nodes fail or the network partitions.

---

1) Core distributed-system building blocks (simple)

Leader election

Purpose: pick one node to coordinate actions (serialize writes, assign leases, manage metadata).
Common algorithms: **Raft** (easier to explain + implement), **Paxos** (theoretical), **Zab** (used by ZooKeeper).
Key ideas (Raft):

- Elect a leader via majority votes.
- Leader accepts client writes, replicates to followers.
- If leader fails or loses majority, a new election happens.

Why leader? simplifies coordination: single sequencer for ordered state changes.

Replication

Purpose: make data durable & available even if nodes die.
Common pattern: **log replication**:

- Leader writes command to its log.
- Leader replicates log entry to majority of followers.
- Once majority ack, leader commits and applies to state machine; then replies to client.
  This yields **strong consistency** (linearizable) when used with majority commit.

Consistency

- **Strong (Linearizability):** operations appear instant and globally ordered — easy mental model, but can be slower and requires majority / synchronous commits.
- **Eventual:** replicas may diverge briefly; converge later — higher availability, lower latency.
  Trade-offs: choose based on needs (locks demand strong consistency).

---

2) Design goals & constraints for a Distributed Lock Service

Functional

- Acquire(key, clientId, timeout) → success/failure
- Release(key, clientId) → success/failure
- Session liveness: if client dies, lock is released automatically (no indefinite blocking)

Non-functional

- Correctness under failures (no two clients hold same lock concurrently)
- Low latency for acquire/release
- Scalable read traffic (many clients wanting different locks)
- Highly available for clients (within the consistency trade-off)

Safety vs Liveness

- Safety (mutual exclusion) is essential — never allow two holders simultaneously.
- Liveness: try to avoid deadlocks and ensure progress, but in network partitions you may sacrifice availability.

---

3) Two common lock models (and why choose one)

1. **Lease-based lock (clock / TTL):** lock has TTL; client must renew lease; if lease expires, lock is free. Simple, but needs careful clock/expiry handling and fencing tokens to prevent stale-holder issues.
2. **Consensus-backed lock (recommended):** implement locks on top of a consensus service (Raft/Quorum) — e.g., ZooKeeper/etcd. Strong guarantees: only one holder, automatic cleanup via ephemeral nodes/sessions.

For interview: recommend consensus-backed ephemeral-node approach (how ZooKeeper does it) or etcd using compare-and-swap (CAS) + leases.

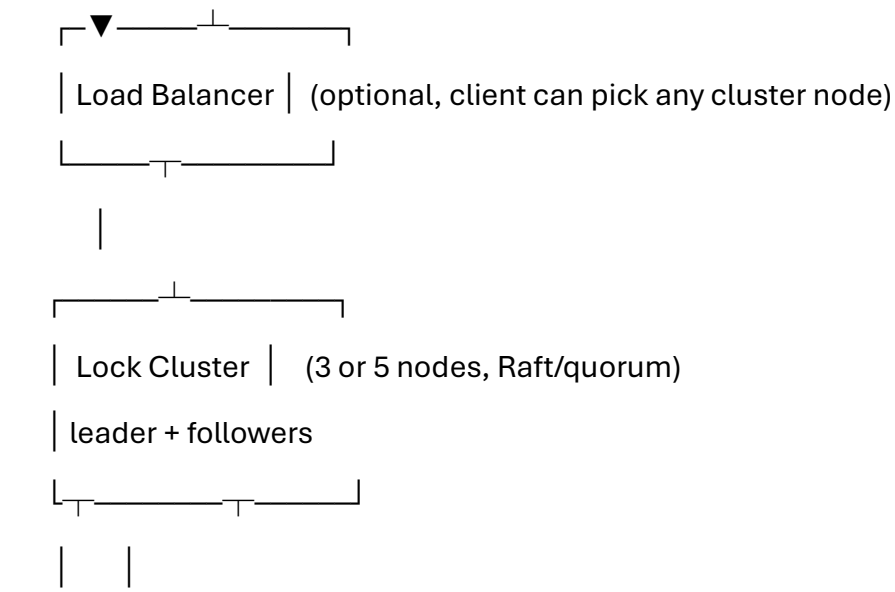
---

4) Architecture (text diagram)

Clients (many)

/ | \

/ | \



- Cluster runs Raft (or ZooKeeper's Zab) and stores lock metadata (e.g., ephemeral keys, sessions).
- Clients contact any node; operations are forwarded to leader for serialization.

---

5) API / semantics (simple)

Acquire(key, clientId, ttlMillis) -> { success: bool, token: string, leaseId: UUID }

Release(key, clientId, token) -> { success: bool }

Renew(leaseId, clientId) -> { success: bool } // optional keep-alive

GetOwner(key) -> { clientId?, leaseExpiry? }

- token (fencing token) returned on successful acquire is monotonic increasing per acquisition (used to avoid stale-holder problems when external resources are involved).
- leaseId maps to client session; session expiry releases all ephemeral locks.

---

6) Implementation approaches (detailed, but simple language)

A — ZooKeeper-style ephemeral nodes (leaderless client interaction)

- Each lock acquisition creates an **ephemeral sequential znode** under /locks/{key}/.
- Clients create ephemeral nodes; client with smallest sequence gets the lock.
- If not smallest, client sets a watch on predecessor node; when predecessor disappears, re-check.
- Ephemeral nature: if client disconnects (session expires), node is removed -> lock freed.

Pros: fairness, good guarantees, auto-cleanup.
Cons: sequential node creation creates many small writes (but ZooKeeper is optimized for this).

B — etcd/Consul with leases + compare-and-swap

- Use a **lease**: acquire a lease via Grant(leaseTTL); then write PUT key -> value with lease using CAS: only succeed if key absent.
- To release: delete key or revoke lease.
- Keep lease alive with periodic heartbeats.
- To avoid split-brain, rely on Raft consensus: only leader can commit writes.

Pros: simple API, backed by strong consensus (Raft), widely used.
Cons: need heartbeats and lease tuning.

---

7) Fencing tokens — preventing stale-holder problems

Important scenario:

- Client A acquires lock and performs actions on external resource (e.g., DB master).
- A fails without releasing; after lease expiry, client B acquires lock and manipulates resource.
- But A restarts and continues work (slow network) → both might act concurrently.

Solution: return **fencing token** (monotonic integer) on acquire. External resource checks token (or token used to set DB cluster epoch). Only operations with token >= current accepted token are allowed. This prevents stale clients from performing actions.

Implement token generation at leader: increment a counter per successful acquire; store in consensus log so strictly increasing.

---

8) Correctness & failure scenarios (and handling)

- **Leader crash:** new election; leader may not have committed last log; consensus ensures only committed entries considered, so locks behave consistently.
- **Client crash:** session TTL triggers; ephemeral key removed; cluster notifies waiting clients.
- **Network partition:** If client loses connectivity to majority, locks held by the partitioned client may be freed after TTL; need fencing tokens to avoid stale actions.
- **Clock skew:** Avoid relying on system clocks for correctness; prefer logical lease management from leader with TTL counters and leases.

---

9) Simple pseudo-code (acquire/release via consensus + lease)

Acquire (simplified):

```
function Acquire(key, clientId, ttl):
  // client sends request to any node -> forwarded to leader
  if key not present:
    token = incrementGlobalToken(key)  // persisted by leader
    store key -> {clientId, expiry = now() + ttl, token}
    return {success: true, token, leaseId}
  else:
    return {success: false, owner = currentOwner}
```

Server-side:

- Leader appends the SET command to Raft log, replicates to majority, commits.
- Followers apply only when committed.

Release:

```
function Release(key, clientId, token):
  if key exists and key.clientId == clientId and token matches:
```

append DELETE to Raft log -> commit

return true

else return false

Renew:

- client must send KeepAlive to renew lease before expiry; leader updates expiry via committed entry or in-memory timer with persisted heartbeat.

---

10) Performance, latency, and scaling

- **Latency:** Acquire/release need a round of consensus commit → ~1-2 RTTs (leader + majority). Expect ~10s–100s ms depending on data center.
- **Throughput:** Cluster throughput limited by leader writes. For many distinct keys, throughput can be improved by partitioning (multiple clusters) or using optimistic locks for non-conflicting keys.
- **Scaling strategy:**
    - For huge scale: partition keys across multiple lock clusters (shard by key hash) — each shard is a separate Raft group.
    - For read-heavy metadata: use followers for reads (but lock operations must be linearized via leader).

---

11) Monitoring & operational concerns

Track:

- Leader election frequency (too many elections → instability)
- Commit latency (Raft)
- Client session count & TTL expirations
- Lock wait times & queue lengths
- Fencing token counter growth

Operational tools:

- Metrics (Prometheus): raft_commit_latency, followers_lag, sessions, watches

- Alerts on high election rate or high follower lag

---

12) Trade-offs & what to tell interviewers

- **Why use consensus (Raft) vs single master?**
    - Consensus: strong safety (mutual exclusion), fault tolerance (survive node failures).
    - Single master: simpler and lower latency but single point of failure.
- **Why TTL/lease?**
    - Ensures automatic cleanup for crashed clients; but requires careful TTL selection.
- **Why use fencing tokens?**
    - Prevents stale clients from acting on external resources after lock expiry.
- **When might we relax linearizability?**
    - For low-critical locks where occasional duplicate work is acceptable: use best-effort locks (e.g., Redis SETNX with TTL) for lower latency.

---

13) Example: Lightweight Redis-based lock (for less critical use-cases)

- SET key value NX PX 30000 → acquire lock if not exists with 30s TTL.
- Release with safe compare-and-del script (only delete if value matches).
- Problems: not safe across Redis failover unless you use Redlock (which is debated). Redlock requires 3+ independent Redis masters and careful implementation.

Mention Redlock briefly as an option but recommend consensus-backed solution for correctness.

---

14) Capacity planning & cost (simple estimate you can speak to)

Assumptions:

- Cluster size: **3 nodes** (minimum for majority, fault tolerance)

- Each node hardware: 4 vCPU, 16 GB RAM
- Each node stores metadata (locks); metadata per lock ~ 200 bytes (key, clientId, token, TTL, overhead)
- Expected max concurrent locks: **1,000,000**

Memory calc:

- 1,000,000 × 200 bytes = 200,000,000 bytes ≈ 190.7 MB
- Add overhead (indexing, logs, replication) → ×4 ≈ 760 MB
- So 16 GB per node is more than enough.

Disk (WAL + logs):

- If we keep 7 days of append-only logs and each lock op is small:
- Estimate 100M ops/day × 200 bytes = 20 GB/day → 7 days ≈ 140 GB (persisted per broker in worst-case)
- But consensus systems often compact logs; real number lower. Use SSDs >= 500 GB for safety.

Cost (AWS example rough):

- **3 × m5.large (2 vCPU, 8GB)** for small load ≈ $70–90/month each → ~$210–270/month.
- For production (4 vCPU, 16GB) ~ $150/month each → ~$450/month.
- Add storage, monitoring, networking → round up. So **small cluster cost ~ $300–1000/month** depending on sizing and region.

You can present both a minimal-cost and production-cost estimate.

---

15) How to explain in an interview (script)

1. **Start with requirements:** mutual exclusion, auto-release on client crash, high availability.
2. **Say chosen approach:** "Use a consensus-backed cluster (Raft) + ephemeral leases so that locks are strongly consistent. This gives us safety and fault tolerance."

3. **Explain flow:** client -> leader -> append to log -> replicate -> commit -> return fencing token / lease.
4. **Cover failure cases:** leader crash → new leader elected; client crash → lease expiry frees lock; show how fencing tokens prevent stale clients from acting.
5. **Mention alternatives & trade-offs:** Redis SETNX for low-latency/best-effort; Redlock; but for correctness use consensus-based approach.
6. **Finish with ops:** monitoring, TTL tuning, sharding for massive scale, rough cost estimate.

---

16) Quick cheat-sheet (bullet list you can memorize)

- Use **Raft** for leader election and log replication.
- Store locks as **ephemeral entries** tied to client sessions or leases.
- Use **fencing tokens** for external resource safety.
- Use **majority commit** for strong consistency.
- **Shard** by key if you need large scale (each shard = separate Raft group).
- **TTL** ensures auto-release; tune to client heartbeat interval.
- For low-criticality: Redis SETNX + TTL (fast but risky).
- Monitor leader elections, commit latency, and session expirations.

Nice — good choice. Below is a **clean, interview-ready note** for **Scalable File/Object Storage** plus a full **Design — File Storage Service (like Dropbox)**. I keep explanations simple, include protocols, design diagram, scaling strategies, consistency/replication, chunking/dedupe/versioning, and example capacity + cost math (with step-by-step arithmetic so you can explain it confidently).

🔹 **Concepts — File / Object Storage (simple)**

Object storage (S3-like)

- Stores *objects* (blob + metadata + key).

- Accessed by unique keys/URLs (e.g., PUT /bucket/key).
- Designed to scale to exabytes, optimized for large volumes, immutable objects (versions optional).
- Good for: photos, backups, videos, files.

File storage (Dropbox-style)

- Provides a file abstraction and metadata (paths, directories, permissions).
- Supports partial updates, sync, sharing, conflict resolution, and per-user namespaces.
- Often implemented on top of object storage (objects hold file chunks) plus metadata DB.

Key differences

- Object: simple key-value for blobs.
- File: richer metadata, directory semantics, sync semantics (clients expect POSIX-like features sometimes).

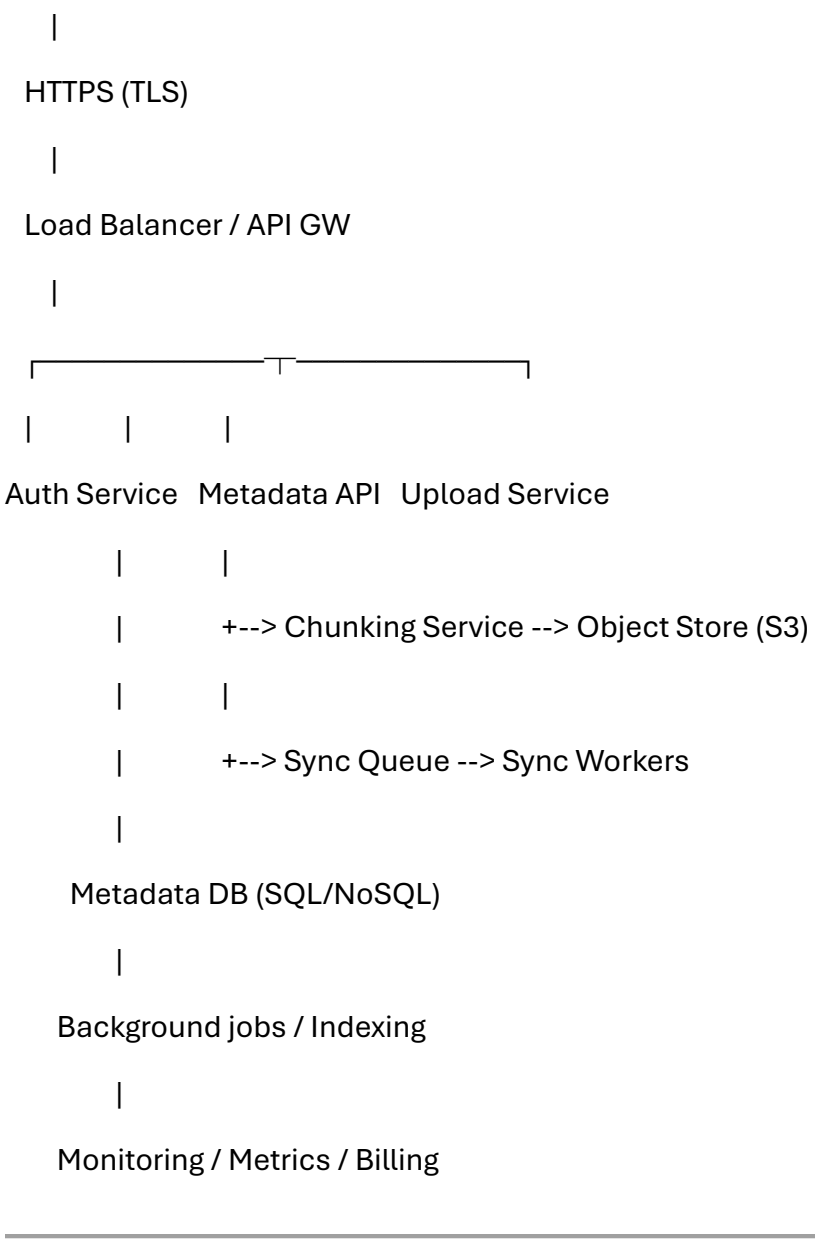
🔹 **Core building blocks (short list)**

- **API Layer / Auth** (HTTPS, OAuth)
- **Metadata DB** (SQL / NoSQL for file/folder metadata, ACLs)
- **Object Store** (S3-compatible / Ceph / custom) for file chunks/versions
- **CDN / Edge cache** for hot downloads
- **Sync Workers / Change-log** for push/pull sync, conflict resolution
- **Chunking + Dedup + Compression** to save space and bandwidth
- **Replication & Erasure Coding** for durability
- **Background jobs**: garbage collection, rebalancing, transcoding, thumbnailing

---

🔹 **High-level architecture (text diagram)**

Clients (desktop/mobile/web)



- **API examples (HTTP/REST)**
  - POST /v1/upload-url → returns presigned upload URL (PUT to object store)
  - PUT /v1/files/{user}/{path} → commit file metadata (references to object chunks)
  - GET /v1/files/{user}/{path} → returns metadata + presigned download URL
  - GET /v1/share/{id} → public download (CDN)
  - GET /v1/changes?cursor=... → sync incremental changes

Protocol: HTTPS for client-to-service. S3 (HTTP/TCP) or internal binary protocol for object store access.

🔹 File storage internals — key ideas

1) Chunking

- Split large files into fixed-size chunks (e.g., 8 MiB) except last chunk.
- Upload chunks independently (multipart upload).
- Metadata stores chunk list + order.

Why: resume uploads, dedupe per-chunk, parallel upload.

2) Content-addressable storage + deduplication

- Store chunks by hash (e.g., SHA-256). If hash exists, do not store duplicate chunk; just reference it.
- Saves storage for users who upload same content.

3) Versioning & immutability

- Keep versions: each commit produces new metadata pointer to chunk list. Old versions kept until GC expiry.

4) Consistency model

- Metadata updates (rename, move, overwrite) must be **strongly consistent** for a single user namespace — use transactional DB (or single-leader per-user shard).
- Object store can be eventually consistent, but metadata points to chunks; commit is atomic (update metadata only after chunk upload checks pass).

5) Sync / Conflict resolution

- Client keeps revision token/cursor. On sync, server returns changes since cursor.
- Conflicts resolved via: last-writer-wins, merge UI, or client-guided resolution. Use vector clocks or versions for stronger resolution.

🔹 Durability & Availability

Options:

- **Replication:** replicate full objects across zones (simple, but storage-costly).
- **Erasure coding:** split object into data+parity shards (e.g., 10+4) — lower storage overhead than replication while still surviving node failures.
- **Region replication:** for geo redundancy.

Trade-off: replication = fast restore & simple; erasure coding = efficient at large scale but cost CPU/IO on repair.

🔹 **Scaling strategies**

- **Horizontal scale API**: stateless API servers behind LB.
- **Metadata DB sharding**: shard by user-id or namespace to scale reads/writes.
- **Object store scaling**: scale storage nodes (S3-like), partition by object key prefixes or consistent hashing.
- **CDN + edge caching**: offload traffic for read-heavy hot objects.
- **Background rebalancer**: move data between storage nodes.

🔹 **Security & privacy**

- TLS in transit.
- Server-side encryption (SSE) and/or client-side encryption.
- Access control lists and signed/time-limited presigned URLs for download/upload.
- Audit logs for activity.

🔹 **Failure modes & mitigation**

- Partial upload: use multipart + manifest + cleanup.
- Metadata DB outage: use read replicas, failover, or degraded mode (read-only).
- Object loss: use replication/erasure-coding + periodic integrity checks.
- Hotspot: use CDN or shard hot files across multiple nodes.

🔹 Cost & capacity examples (step-by-step arithmetic)

We'll do a worked example so you can explain capacity & cost in an interview.

Assumptions (example scenario)

- **Users:** 1,000,000 (one million) users.
- **Average storage per user:** 5 GB (gigabytes) each.
- We plan to **replicate 3×** (triple copy) for durability OR use equivalent across erasure coding → for simplicity we assume 3× raw replication.
- **Object store price assumption:** $0.02 per GB-month (example; use your cloud's published price in real interview).
- **Egress cost assumption:** $0.09 per GB (for CDN / data transfer out).
- **Active daily download:** 1% of users download an average 50 MB per day.

I'll compute totals step-by-step.

A — Total raw stored bytes (single copy)

1. Per user = 5 GB.
2. Users = 1,000,000.

Multiply:

- 5 × 1,000,000 = 5,000,000 (GB)

So **single-copy storage = 5,000,000 GB**.

In petabytes:

- 1 PB = 1,000,000 GB (decimal).
- 5,000,000 GB ÷ 1,000,000 = 5 PB.

So **single-copy = 5 PB**.

B — With 3× replication

- 5,000,000 GB × 3 = 15,000,000 GB.

Convert to PB:

- 15,000,000 ÷ 1,000,000 = 15 PB.

So **replicated storage ≈ 15 PB**.

C — Monthly storage cost (estimate)

Using $0.02 per GB-month (assumption):

- Single copy cost = 5,000,000 GB × $0.02/GB =
    - Multiply: 5,000,000 × 0.02 = 100,000
    - (Because 5,000,000 × 2 = 10,000,000; then divide by 100 → 100,000)

So **single-copy ≈ $100,000 / month**.

- For 3× replication: 15,000,000 GB × $0.02 =
    - 15,000,000 × 0.02 = 300,000

So **replicated cost ≈ $300,000 / month**.

Tell interviewer: "These are ballpark numbers — actual cost depends on storage class (frequent vs infrequent), discounts, and erasure coding."

D — Daily egress estimate (active downloads)

Assumptions:

- Active daily users = 1% of 1,000,000 = 10,000 users.
    - Multiply: 1,000,000 × 0.01 = 10,000.
- Average download per active user = 50 MB (megabytes).

Total daily download (in MB):

- 10,000 users × 50 MB = 500,000 MB.

Convert MB → GB:

- 1 GB = 1024 MB (if using binary), but for cost we use decimal approx 1000 MB ≈ 1 GB for simplicity. Clarify your choice. Using decimal:
    - 500,000 MB ÷ 1000 = 500 GB/day.

Monthly egress:

- 500 GB/day × 30 = 15,000 GB/month = 15 TB/month.

Egress cost at $0.09/GB:

- 15,000 GB × $0.09 =

- 15,000 × 0.09 = 1,350

So **egress ≈ $1,350 / month**.

(If you use CDN with cheaper edge cost or committed discounts, this reduces.)

🔹 **Example storage optimizations to lower cost**

- **Deduplication** reduces stored bytes (especially effective for backups/photos). If dedupe saves 30%, storage drop from 5 PB → 3.5 PB.
- **Compression**: store compressed objects (images already compressed; text compresses well).
- **Storage classes**: Move cold data to cheaper tiers (e.g., low-frequency or archival) with lifecycle policies.
- **Erasure coding** vs replication: erasure coding with 10+4 reduces overhead vs 3× replication — can cut storage overhead substantially.

🔹 **Operational & design talking points (what interviewers expect)**

- **Why object store + metadata DB?**
  Object store is scalable for blobs; metadata DB provides quick lookups, ACLs, directory semantics. Separating concerns lets each layer scale independently.
- **Why chunking & content-addressable?**
  Enables parallel upload, resume, dedupe, and small-change updates without rewriting whole file.
- **Why presigned URLs?**
  Offload bandwidth: clients upload/download directly to object store (S3) using temporary signed URLs, reducing API server load.
- **How to ensure consistency?**
  Make metadata update atomic: finalize object only after all chunks exist; update metadata in a single DB transaction pointing to chunk list. For multi-region, use per-region primary + async replication or strong consensus per-user shard.
- **How to scale metadata DB?**
  Shard by user-id or namespace; keep hot-user caches in Redis; use secondary indexes for queries.

- **How to handle offline clients & sync?**
  Client maintains local state + change journal; on reconnect, client uploads changes then performs 3-way merge using server version plus client edits.
- **How to prevent data loss?**
  Use replication/erasure coding, periodic integrity checks, and backups for metadata DB.
- **Security & privacy:** TLS + encryption-at-rest + per-object ACL + audit logs.

🔹 **Short checklist to present in interview (script)**

1. **Start with requirements** (support upload/download, sync, share, scale to X users).
2. **State high-level design**: API → metadata DB → object store; CDN for reads. Draw diagram.
3. **Explain internals**: chunking, hash-based storage, multipart uploads, presigned URLs.
4. **Durability**: replicate or erasure code; periodic checks.
5. **Scaling**: shard metadata, horizontal app servers, CDN; background rebalancer.
6. **Costs**: show simple math (users × avg size × replication × $/GB).
7. **Edge cases & trade-offs**: conflict resolution, eventual vs strong consistency, TTL/garbage collection.
8. **Monitoring**: metrics on storage, bandwidth, request latency, error rate.

Got it — here's a **clean, simple, interview-ready note** you can read out loud and draw on a whiteboard for **Scalable File/Object Storage** plus a compact **Design — File Storage Service (Dropbox-style)**. I kept it straight to the point, with clear reasons for each choice, a high-level diagram, API surface, consistency points, scaling steps, and a step-by-step cost + capacity example you can explain confidently.

---

1) Concepts — File storage vs Object storage (simple)

- **Object storage (S3-like)**
    - Stores *objects*: key -> (blob + metadata).
    - Good for large scale, immutable blobs (photos, video, backups).
    - Access via HTTP APIs (S3 REST). Highly scalable, eventually consistent on some operations (depends on provider).
- **File storage (Dropbox-style)**
    - Provides file/directory semantics, sync, versioning, ACLs.
    - Usually built *on top* of object storage for blobs + a metadata DB for file tree, versions, permissions.
    - Needs stronger metadata consistency for user experience (e.g., rename, move, conflict resolution).

Short interviewer answer: use **object store** for blobs and a **metadata DB** for directory/file state.

---

2) High-level requirements (what we must support)

Functional

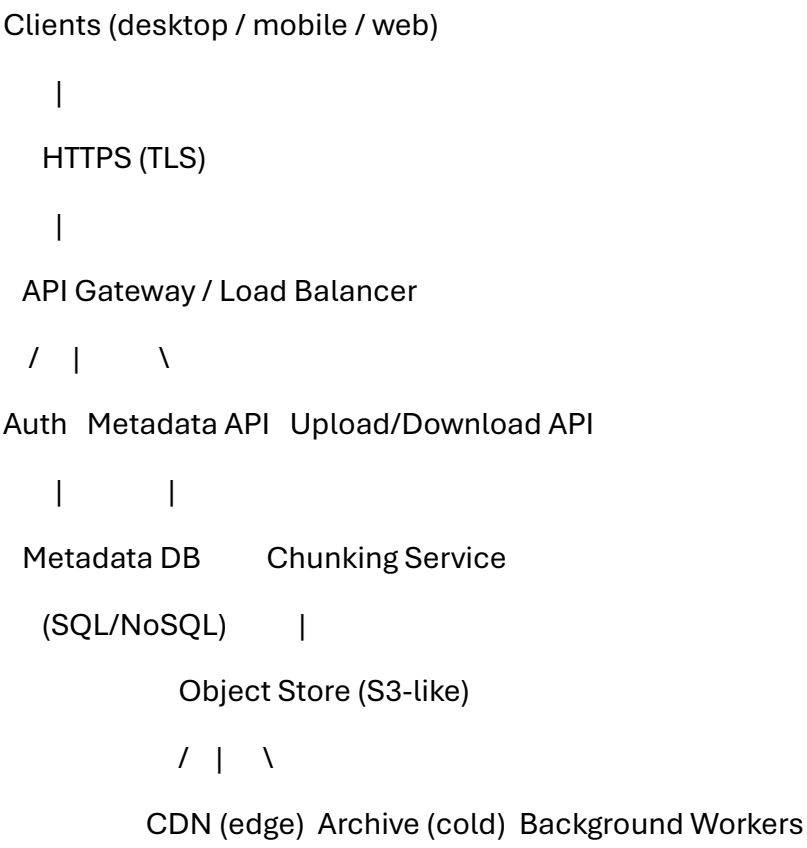
- Upload/download files (large files, resumable)
- File metadata: path, owner, permissions, versions
- Sync (client can list changes)
- Shareable links, thumbnails

Non-functional

- Durable (no data loss), highly available, low latency for reads
- Scale to petabytes
- Cost-effective (cold/hot tiers)
- Secure (auth, encryption, presigned URLs)

---

3) High-level architecture (text diagram)



Flow highlights:

- Client asks for presigned upload URL -> uploads directly to object store (reduces LB bandwidth).
- App commits metadata after successful chunk uploads.
- Downloads served via CDN with presigned download URLs.

---

4) Key design components & why

- **API Gateway (HTTPS)** — central auth point, rate limiting. Use HTTPS for security and wide client support.
- **Metadata DB (SQL or NoSQL)** — stores file tree, versions, ACLs, chunk lists. Use SQL for strong transactions (single-user ops), or NoSQL sharded per user for scale.
- **Object Store (S3-compatible)** — stores chunks/objects. Immutable objects simplify concurrency and recovery.

- **Chunking + Multipart Upload** — split files (e.g., 8 MB chunks) to enable resumable uploads and parallelism.
- **Content-addressable storage + dedupe** — store chunks by hash (SHA-256); if same hash exists, link rather than store again.
- **CDN / Edge** — cache hot downloads; reduces egress and latency.
- **Background workers** — dedupe, transcode, thumbnail, lifecycle (move cold data to archive).
- **Replication / Durability** — either multi-AZ replication or erasure coding for lower storage overhead and high durability.

---

5) API surface (simple)

- POST /v1/upload-url → returns presigned PUT URL (client uploads directly to object store).
- POST /v1/commit → commit file metadata (list of chunk hashes, size, owner, version).
- GET /v1/files/{user}/{path} → get metadata + presigned download URL.
- GET /v1/changes?cursor=... → get changes since cursor for sync.
- POST /v1/share/{path} → create shareable link.

Why presigned URLs? offloads traffic to object store, lowers server bandwidth and cost.

---

6) Data model (minimal)

Metadata table (relational or document):

- file_id (PK), user_id, path, version_id, size, chunk_list (ordered hashes), created_at, acl, is_dir

Chunk store (object store):

- Key = sha256(chunk) or bucket/prefix/sha256
- Object metadata contains reference counts for GC

---

7) Consistency & atomicity

- **Metadata updates must be atomic**: only update commit record after all chunks exist (two-phase: upload chunks → commit metadata transaction).
- **User-level strong consistency**: use per-user sequencing or single-leader per user-shard to ensure operations like rename/overwrite are linearizable for that user.
- **Global object store** can be eventually consistent; metadata points to chunks and is authoritative for visible state.

---

8) Durability & replication options

- **Replication (3× copies)** — simple, fast reads, higher storage cost.
- **Erasure coding (e.g., 10+4)** — stores data across N shards + parity, lower cost (overhead < 3×), but more CPU on reads/writes and more complex rebuilds.
- Choose erasure coding for cold/large-scale storage; replication for hot objects and simpler recovery.

---

9) Scaling strategy

- **API/Upload servers**: stateless, auto-scale behind LB.
- **Metadata DB**: shard by user_id; use read replicas for read-heavy listing operations.
- **Object store**: scale storage nodes or use managed S3 (virtually infinite).
- **CDN**: cache hot downloads; reduces load on object store and egress cost.
- **Background workers**: scale for dedupe, transcoding, integrity checks.

---

10) Security & access control

- HTTPS for transport.
- Authentication: OAuth / JWT tokens.
- Authorization: ACL per file.
- Presigned URLs expire — limit exposure.

- Encryption at rest (SSE) + optional client-side encryption.

---

11) Failure modes & mitigation

- **Partial upload**: keep multipart manifest; clean up orphan chunks via background GC with reference counting.
- **Metadata DB outage**: serve read-only from replicas; queue writes for failover.
- **Object loss**: use replication/erasure coding and periodic checksums.
- **Hotspot**: use CDN + shard metadata to distribute load.

---

12) Cost & capacity calculation — step-by-step example (say this aloud)

Assumptions (example scenario you can present):

- **Users** = 1,000,000
- **Avg storage per user** = 5 GB
- **Replication factor** = 3× (simple replication)
- **Object store price** = $0.02 per GB-month (example)
- **Active daily downloads** = 1% users daily, average download 50 MB
- Use decimal GB (1 GB = 1000 MB) for cost calc (state this explicitly).

Step A — total single-copy storage:

- Per user = 5 GB
- Users = 1,000,000
- Multiply: 5 × 1,000,000 = 5,000,000 GB (single copy).
  (5 times 1,000,000 equals 5,000,000)

Step B — replicated storage (3×):

- 5,000,000 × 3 = 15,000,000 GB
  (5,000,000 times 3 equals 15,000,000)

Step C — monthly storage cost:

- Price = $0.02 per GB-month

- Cost = 15,000,000 × 0.02 = $300,000 per month
  (15,000,000 × 2 = 30,000,000; divide by 100 => 300,000)

Step D — daily egress (downloads):

- Active daily users = 1% of 1,000,000 = 10,000
  (1,000,000 × 0.01 = 10,000)
- Avg download = 50 MB → per day total MB = 10,000 × 50 = 500,000 MB
  (10,000 times 50 equals 500,000)
- Convert to GB (decimal): 500,000 MB ÷ 1000 = 500 GB/day
- Monthly egress = 500 × 30 = 15,000 GB/month
- Egress cost @ $0.09/GB = 15,000 × 0.09 = $1,350 per month
  (15,000 × 9 = 135,000; divide by 100 => 1,350)

Total rough monthly cost (storage + egress) ≈ $300,000 + $1,350 ≈ **$301,350** (ignoring compute, CDN, DB, and discounts).

Always say: “ballpark, depends on storage class, cloud discounts, and use of erasure coding.”

---

13) Optimization levers to mention in interview

- **Deduplication** (content-addressable chunks) — cuts storage if duplicates common.
- **Compression** — especially for textual data.
- **Lifecycle policies** — move cold data to cheaper storage (archive).
- **Erasure coding** — reduces storage overhead vs 3× replication.
- **CDN caching** — reduces egress and latency.
- **Tiered pricing / reserved capacity** — lower cost at large scale.

---

14) How to present this in a 2–3 minute interview answer (script)

1. Start: “I’d separate blobs and metadata — object store for chunk storage and a metadata DB for file tree, ACLs, and versions.”

2. Sketch diagram: API → metadata DB → object store (with CDN).
3. Explain upload flow briefly: presigned URL → direct upload (multipart chunks) → commit metadata.
4. Mention consistency: “metadata commit is atomic and authoritative; object store only contains chunks.”
5. Cover durability: “use replication or erasure coding; pick based on cost vs rebuild speed.”
6. Finish with capacity/cost example: run through the arithmetic quickly to show you can size the system.

Nice — here’s a compact, interview-ready **End-to-End design** cheat sheet plus a complete **Design Mini: Instagram Feed / Twitter Timeline** you can present on a whiteboard or explain in 3–5 minutes. I keep language simple, include trade-offs, bottlenecks, scaling/estimation math step-by-step, and a recommended architecture (push/pull hybrid).

End-to-End Design — short checklist

- Start with **requirements** (functional + non-functional).
- Pick **data model** and **API surface**.
- Decide **read/write pattern** (read-heavy vs write-heavy).
- Choose **fan-out strategy** (push, pull, hybrid).
- Add **caching**, **indexing**, **sharding**, **replication**.
- Plan **delivery & ranking** (personalization / ML).
- Address **durability, consistency, rate limiting, retries**.
- Identify **bottlenecks** and mitigations.
- Provide **capacity estimates** and cost ballpark.
- End with monitoring, testing, and incremental rollout plan.

Design Mini — Instagram Feed / Twitter Timeline

1) Requirements

Functional (core)

- User A posts a tweet/photo.
- Followers of A see that item in their timeline (feed).
- Support home feed (personalized) and user profile feed (user's own posts).
- Like/retweet/comment interactions (counts).
- Real-time or near-real-time delivery desirable.

Non-functional

- Millions of users, billions of feeds reads/day (read-heavy).
- Low read latency (<200ms).
- High availability and durability.
- Support ordering & ranking (recency + relevance).
- Reasonable cost.

Optional / Nice-to-have

- Personalized ranking (ML).
- Caching of top/viral posts.
- Offline read / infinite scroll with pagination.

---

2) High-level choices & trade-offs

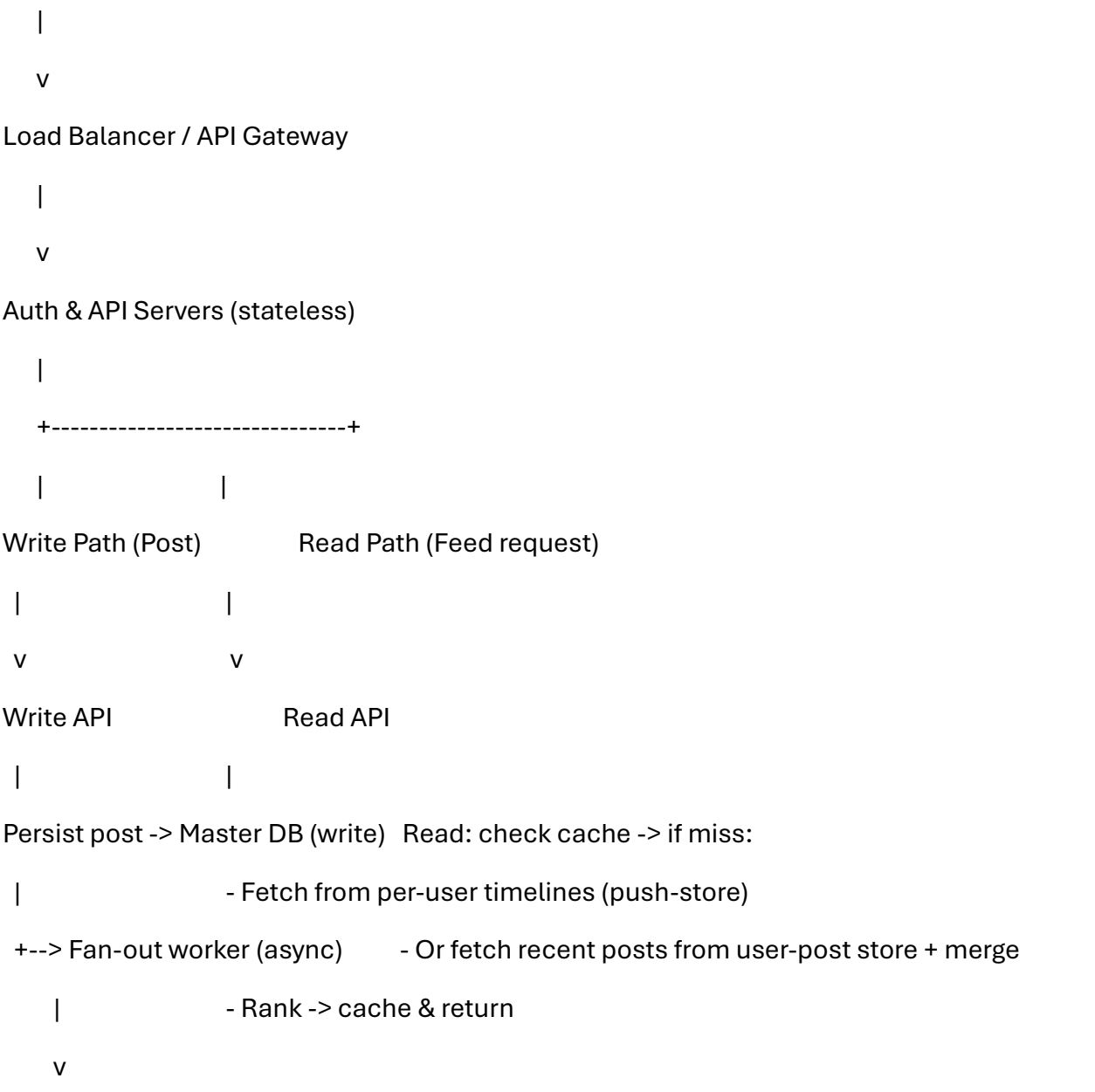
- **Push (fan-out-on-write)**: when a user posts, push the post ID into each follower's feed shard. Pros: reads are fast. Cons: expensive if poster has millions of followers (celebrity problem).
- **Pull (fan-out-on-read)**: on read, fetch recent posts from followed users and merge-sort. Pros: cheap writes, consistent; Cons: high read latency and compute for heavy readers.

- **Hybrid** (recommended): push for normal users; for very high-fanout users (celebrities), use pull or special handling (store at origin and fetch at read time). This balances costs.

Trade-offs: push reduces read latency and CPU at read time at cost of write amplification. Pull reduces write cost but increases read latency and compute.

---

3) High-level architecture (text diagram)



Message Broker (Kafka)

|

v

Fan-out workers -> write to per-user timeline store (Redis / DynamoDB)

|

v

Analytics / Search / ML ranking (offline)

Components:

- **Post Store (append-only)**: durable storage for posts (S3-like or DB), sharded by user-id.
- **Timeline Store (per-user feed)**: fast key-value store (Redis or DynamoDB) that stores list of postIDs per user (ordered). Used for push-read.
- **User-Post Index**: service to read recent posts by a given user (for pull).
- **Message Broker (Kafka)**: durable event stream for fan-out and async processing.
- **Fan-out Workers**: consume events, push postIDs into followers' timeline lists.
- **Cache / CDN**: cache hot feeds and media.
- **Ranking Service**: re-ranks or scores items (online or cached results).
- **Metrics & Monitoring**.

---

4) Data model (simplified)

- **Post**: {post_id, author_id, content, media_url, created_at, metadata}
- **UserFollowers**: stored per user; follower list used by fan-out workers
- **Timeline (per user)**: ordered list of post_ids (e.g., Redis list or DynamoDB sorted set)
- **UserPosts**: per-user chronological posts (append-only)
- **Interactions**: counters for likes/comments (eventually consistent counters)

---

5) Read & write flows

Write (Post)

1. Client POST /posts -> API server validates and stores post to Post Store (durable).
2. API publishes post_created event to Kafka (with post metadata).
3. Fan-out workers consume event, read follower list (sharded) and:
    - For each follower: push post_id to follower's timeline store (Redis list or DB).
    - If follower timeline > N (e.g., keep last 1000), trim.
    - For high-fanout authors, skip full push and mark post as “pull-only” or push to only top followers.
4. Optionally update caches & analytics.

Read (Get home feed)

1. Client GET /feed?page=X -> API server checks timeline cache (Redis).
2. If cache hit: return postIDs → fetch posts (batched) from Post Store or cache.
3. If miss: fetch from timeline store (DB) or use pull strategy: fetch recent posts from followees, merge-sort by score/time, apply ranking, cache result.

---

6) Ranking & personalization

- Basic ranking: recency + simple weight (engagement).
- Advanced: ML model scores per-user/post (offline training + online features).
- Use a **ranking service** that takes candidate set (from timeline store or pull) and returns ordered subset.
- Store precomputed scores for hot items to reduce latency.

---

7) Bottlenecks & mitigations

- **Fan-out storm** (celebrity posts): mitigate by hybrid push/pull; precompute only for recent followers; use special tier for celebrities (eg. serve their posts from user-post store on read).

- **Follower list read**: store follower lists sharded and cached; avoid reading huge follower lists synchronously.
- **Timeline store size**: trim timelines to fixed length; archive older posts.
- **Post fetching latency**: batch post fetches, use caching, and CDN for media.
- **DB write throughput**: shard by user-id, use write-optimized stores (Cassandra/DynamoDB) for append-only data.

---

8) Capacity & cost estimation (step-by-step example you can explain)

Assumptions (example for interview):

- **Users** = 100M total
- **Daily active users (DAU)** = 10% = 10M
- **Avg posts per DAU per day** = 0.5 → posts/day = 10M * 0.5 = 5M posts/day
- **Average post size (metadata + reference)** = 1 KB (media stored separately in object store)
- **Reads per DAU per day** = 50 feed requests = 10M * 50 = 500M feed reads/day
- **Average feed items per read** = 30 (we fetch 30 postIDs and batched post metadata)

Storage for posts (metadata) per month

- Daily posts = 5M → monthly ≈ 150M posts
- Metadata storage = 150M * 1 KB = 150,000,000 KB ≈ 150,000 MB ≈ 150 GB/month (metadata)
  (150,000,000 KB ÷ 1000 = 150,000 MB; ÷ 1000 = 150 GB)
- Media (photos/videos) stored in S3 separately; if avg media size 2 MB and 30% posts have media:
  - Media objects/day = 5M * 0.3 = 1.5M
  - Daily media size = 1.5M * 2 MB = 3,000,000 MB = 3000 GB = 3 TB/day
  - Monthly ≈ 90 TB for media

Timeline storage (push store)

- We keep last 1000 postIDs per user (list length = 1000)

- PostID size ≈ 8 bytes
- Per-user timeline = 1000 * 8 B = 8000 B ≈ 8 KB
- For DAU 10M users (assuming maintain for all active users) = 10M * 8 KB = 80,000,000 KB = 80,000 MB ≈ 80 GB
- Add replication & overhead → ×3 → ≈ 240 GB

Traffic estimate (feed reads)

- Feed reads/day = 500M requests
- Each request fetches 30 post metadata (if cached): metadata total payload ≈ 30 * 1 KB = 30 KB
- Total bytes/day = 500M * 30 KB = 15,000,000,000 KB ≈ 15,000,000 MB ≈ 15,000 GB = 15 TB/day
- Use CDN + cache to reduce origin egress significantly (e.g., only 10% hits origin) → 1.5 TB/day origin egress

Cost ballpark (very rough)

- Object storage for media (90 TB/month) at $0.02/GB = 90,000 GB * $0.02 = $1,800/month
- Timeline store (Redis/DynamoDB) memory ~ 240 GB → managed Redis ~ cost varies (rough $1k–2k/month)
- Compute & DB & Kafka & network → add several thousand dollars. Total could be in low tens of thousands monthly for this scale; emphasize these are rough.

(When speaking: always say *assumptions* and show arithmetic steps.)

---

9) Consistency, correctness, and failure modes

- Feed is allowed to be **eventually consistent** for new posts (user may see post with slight delay).
- Ensure **idempotent** writes in fan-out workers; use message broker with at-least-once semantics + de-dup via postIDs stored as sets.
- Recover from worker failure by replaying Kafka topic.
- Rate-limiting: prevent abusive readers and poster spamming.

---

10) Metrics & monitoring (what interviewers love)

- Read latency (p50, p95, p99)
- Feed generation time
- Kafka lag (fan-out backlog)
- Time-to-delivery for posts (publish→visible)
- Error rates, 5xx responses
- Hot-user load & queue lengths

---

11) Security & privacy

- Authentication: OAuth/JWT for API.
- Authorization: ACLs for private accounts.
- Protect media & presigned URLs with short TTL.
- Rate-limit endpoints per-user IP & client.

---

12) 2-minute interview script (memorize & deliver)

1. **Requirements:** "We need a read-heavy timeline: posts, follower feeds, low-latency reads, personalized ranking."
2. **High-level design:** "Use an append-only Post Store for durability, a Timeline Store (Redis/DynamoDB) for fast per-user feed (push-on-write), and Kafka for fan-out. Use hybrid push/pull: push to normal followers and pull for very high-fanout authors."
3. **Read flow:** "On read, check timeline cache; if miss, retrieve postIDs from timeline store, batch-fetch post metadata, run ranking, return. Cache results."
4. **Scaling & bottlenecks:** "Fan-out storm is main issue — handle with hybrid strategy, rate limit, and special-case celebrities. Shard follower lists and timeline storage by user-id."
5. **Sizing example:** (quickly give one two-line calculation: e.g., 5M posts/day → ~150M/month → X TB media/month — show you can estimate).

6. **Monitoring & ops:** “Monitor Kafka lag, p99 latency, and election counts; use retries and idempotence.”

1) Design: E-commerce System (like Amazon product catalog + orders + checkout)

1.1 Requirements (must-have)

Functional

- Browse product catalog (search, categories, product detail)
- Add to cart, checkout, place order, payment
- Inventory tracking (prevent oversell)
- Order history & status tracking
- Basic recommendations (“similar items”)
- Reviews & ratings

Non-functional

- Highly available, low read latency for product pages
- Strong inventory correctness (no double-selling)
- Handle traffic spikes (sales)
- Scalable to millions of users & product SKUs
- Secure payment processing (PCI concerns)

1.2 High-level APIs

- GET /products?query=...&page=... — search / browse
- GET /products/{sku} — product detail (price, stock, images)
- POST /cart/{user} — add item / update cart
- POST /checkout — create order (validate stock, reserve inventory)
- GET /orders/{orderId} — order status
- POST /payments — integrate with payment gateway (tokenized)
- POST /inventory/reserve / POST /inventory/commit — inventory ops (internal)

1.3 Data model (simplified)

- **Product**: {sku, title, desc, price, images[], categories[], attributes, created_at}
- **Inventory**: {sku, warehouse_id, available_qty, reserved_qty}
- **Cart**: {user_id, items[{sku, qty, added_at}], last_updated}
- **Order**: {order_id, user_id, items, total_amount, payment_status, fulfillment_status, created_at}
- **User**: {user_id, address[], payment_methods[]}
- **Review**: {sku, user_id, rating, comment, created_at}

1.4 High-level architecture (text diagram)

Clients (web/mobile)

|

CDN (images/static) <— product images, static pages

|

API Gateway / LB

|

Auth & API Servers (stateless)

| \

| --> Search Service (Elasticsearch)

|

Cart/Order Service <---> Payment Gateway (3rd party)

|

Inventory Service (strong correctness)

|

Order DB (SQL for ACID) Product Catalog (NoSQL or RDB)

|

Warehouse / Fulfillment / Shipping

Key components:

- **API Gateway / LB**
- **Catalog Service** (read-heavy) — backs product pages, cached heavily (Redis / CDN)
- **Search Service** — Elasticsearch for full-text search & filters
- **Cart Service** — session-like, can use Redis
- **Checkout & Order Service** — ACID requirements — use relational DB (e.g., Postgres)
- **Inventory Service** — either centralized or per-warehouse; needs strong consistency to avoid oversell
- **Payment Service** — external gateway; tokenized payments
- **Recommendation Service** — offline ML + cached results
- **Background workers** — reconciliation, order fulfillment, email
- **Message broker** (Kafka/RabbitMQ) — decouple operations: inventory updates, notifications

1.5 Important flows

Browse / Product page

1. Client requests product → CDN/Redis cache check.
2. Cache miss → Catalog DB → return + populate cache.
3. Images served from CDN.

Checkout (critical path)

1. User submits order → API server validates cart, price calculations.
2. **Inventory reservation**: Atomically reserve stock (decrement available, increase reserved) via Inventory Service.
    - Use DB transaction or distributed lock/compare-and-swap.
3. Create Order record (status: PENDING) in Order DB (transaction).
4. Call Payment Gateway with token → on success, mark payment confirmed.

5. Commit inventory (deduct reserved -> sold) and update order status to CONFIRMED.
6. Publish events for fulfillment & notifications.

Important: Break the strong part (inventory, payment, order creation) into transactional steps or use two-phase commit pattern / Saga to maintain consistency while remaining scalable. Saga pattern (compensating actions) is common.

1.6 Inventory correctness — options

- **Single authoritative inventory service** (centralized DB) → simplest but a scale/availability bottleneck.
- **Distributed per-warehouse inventory** with global view + routing to appropriate fulfillment center.
- Use **optimistic locking (version/CAS)** or DB-level transactions to reserve stock; for high concurrency, use in-memory counters (Redis with Lua script for atomic check+decrement + persistence) + asynchronous reconciliation.

1.7 Scaling & partitioning

- **Read-heavy** product pages: use caching (Redis + CDN). Product catalog can be sharded by SKU prefix or stored in NoSQL with read replicas.
- **Orders & inventory**: partition by user_id or sku / warehouse_id to distribute load. Order DBs can be sharded by order_id ranges or user region.
- **Search**: Elasticsearch clusters, index product fields and attributes.
- **Message queue** (Kafka) for async fan-out (email, analytics, inventory updates).

1.8 Bottlenecks & mitigations

- Checkout spike (e.g., Black Friday): use queuing (API throttling), pre-reserve flows (limit concurrent checkouts), backpressure.
- Hot SKUs: cache misses → use warm caches, serve degraded results (e.g., prevent exact inventory queries via cache).
- Payments: external gateway latency → async confirmation with order held for short period.

1.9 Data consistency & failure handling

- Use **idempotency tokens** for checkout/payment to avoid duplicates.

- Use **Sagas** for distributed transactions: e.g., if payment fails after reservation, release reserved inventory.
- Keep audit logs for reconciliation.

1.10 Capacity & example estimate (simple)

Assume:

- 1M daily active users, 100K checkouts/day, 10M product page views/day.
- Product metadata size negligible; cache needs e.g., 50GB Redis to store hot product pages.
- Inventory DB size depends on SKUs — if 1M SKUs, per-SKU metadata ~1KB → ~1GB storage.

Costs vary widely; give ballpark compute & storage numbers when asked.

1.11 Monitoring & ops

Metrics: checkout latency, payment failures, inventory mismatch rates, cache hit ratio, order throughput. Alerts on high payment failures or high inventory reconciliation errors. Use centralized logging, tracing.

1.12 Follow-ups to prepare for interviewer

- Deep-dive into Inventory Service implementation (Redis vs SQL, distributed locking).
- How to handle eventual consistency for analytics/orders.
- How to implement returns/refunds.
- Fraud detection & payment security.

---

2) Design: Uber-like Ride-Hailing System (core features: matching riders & drivers, trip lifecycle)

2.1 Requirements (must-have)

Functional

- Rider requests ride; system finds nearby drivers and notifies them.
- Drivers accept/reject; assignment & real-time updates.

- Start/stop trip, fare calculation, rating
- Live tracking (map updates)
- Support surge pricing

Non-functional

- Low latency for matching (seconds or less)
- Scalable to millions of users & drivers
- Highly available; tolerant to network partitions
- Location updates are frequent — efficient ingestion

2.2 High-level APIs

- POST /rides/request — rider requests a ride (pick-up, drop-off)
- GET /drivers/nearby?lat=X&lng=Y&radius=... — candidate driver list (internal)
- POST /rides/assign — assign driver (internal)
- POST /rides/{id}/start / /end — trip lifecycle
- WebSocket or push endpoints for real-time updates

2.3 Data model (simplified)

- **Driver**: {driver_id, current_location, status (available/on_trip), car_info}
- **Rider**: {user_id, payment_methods}
- **Ride Request**: {request_id, rider_id, pickup, dropoff, timestamp, status}
- **Trip**: {trip_id, driver_id, rider_id, route, start_time, end_time, fare}
- **Pricing**: surge multipliers, base fare rules

2.4 High-level architecture (text diagram)

Mobile Clients (rider/driver)

| (persisted websockets/long-poll)

API Gateway / LB

|

Auth & API Servers

|

+--------------------------+

| Real-time Location Ingest| <-- drivers send frequent updates

+--------------------------+

|

Geospatial Index (in-memory) <- Redis / ElasticGeo / QuadTree

|

Matching Service <--> Pricing Service (surge)

|

Dispatch / Assignment Queue (Kafka)

|

Notification Service (pushes to driver/rider)

|

Trip Management (persist to DB)

|

Billing / Payments

Key components:

- **Location ingestion**: handle high-volume location updates from drivers (e.g., every 1–5s)
- **Geospatial index**: for querying nearby drivers (Redis GEO or custom in-memory index)
- **Matching / Dispatch service**: apply business rules (proximity, ETA, driver preferences)
- **Real-time channel**: WebSockets or push notifications to drivers/riders for assignment
- **Pricing service**: calculates fare and surge using demand/supply signals from streams

- **Trip & Billing services**: persistent storage, fare settlement, receipts

2.5 Core flows

Ride request & match (simple)

1. Rider requests ride → API server writes request to Ride Request DB and publishes an event.
2. **Matching service** queries geospatial index for nearby available drivers (within radius / k nearest).
3. Matching picks top N drivers and sends notifications (push) in parallel (or sequential ring).
4. Driver accepts → system assigns and locks the trip for that driver (use atomic claim to avoid double-assign).
5. Driver arrives → start trip, then end trip; billing post-processing calculates fare & charges rider.

Location update

- Driver app sends location every 1–5s → API gateway → location ingest nodes → update in-memory geospatial index (and occasionally persist to DB).
- To reduce write pressure: sample and aggregate, or use smaller intervals for active drivers only.

2.6 Matching logic & consistency

- For fairness and correctness, use **atomic assignment**: when driver accepts, perform a CAS or write to DB with trip_id and check if still available.
- Avoid race by locking driver or ride in DB or using distributed lock.

2.7 Scaling & partitioning

- **Geospatial partitioning**: partition drivers by geo-tiles or regions (shard by lat/lng) to localize matching traffic.
- **Matchmaking scale**: multiple match services per region; use consistent hashing or region coordinator.
- **Location ingestion**: horizontally scalable ingest fleet; aggregated updates to geospatial index.

- **Data storage**: trip data and billing done in sharded SQL DB; operational state (driver status) in fast in-memory store (Redis).

2.8 Bottlenecks & mitigations

- Hot regions: surge & autoscale more matchers, or degrade matching radius temporarily.
- Location churn: rate limit updates, downsample when driver stationary.
- Assignment storms: limit retries, ring-of-notify pattern, or pre-warm notifications.
- Network latency: use edge servers / regional data centers.

2.9 Fault tolerance & failure modes

- If a driver doesn't respond in time, mark as unavailable and try next.
- If matching service or location index fails, fall back to broader radius or cached driver list.
- For payments: offload to 3rd-party processors with async confirmation & idempotency tokens.

2.10 Real-time considerations

- Use **WebSocket** or **MQTT** for low-latency bi-directional channels (driver updates, assignment).
- For mobile push fallback, use FCM/APNs.
- Manage connection state: persistent connections should be sticky to same region for reduced latency and simplified state.

2.11 Estimations & sample numbers

Assume city with:

- 100k active drivers, each sending location every 5s → 20 updates/sec/driver? No — 1 per 5s = 0.2 updates/sec.
    - Total updates/sec = 100k * 0.2 = 20k updates/sec
- Each update ~ 100 bytes → 20k * 100B = 2,000,000 B/s ≈ 2 MB/s ~ 172.8 GB/day ingest. (You'd explain arithmetic)
- Matching requests: if 10k ride requests/hour → ~2.8 requests/sec — small; scale across many cities.

Estimate compute & memory for geospatial index: store driver ID + lat/lng + status ~ 40 bytes per driver => 100k * 40B = 4MB in-memory — trivial per region. But real systems store more metadata and millions of drivers across globe.

2.12 Monitoring & SLOs

Metrics: assignment latency, driver acceptance rate, time-to-pickup, location update lag, connection drop rate, failed payments, surge accuracy.

2.13 Security & privacy

- Secure location transmissions (TLS)
- Limit location history retention for privacy
- Auth & token expiry for driver apps

2.14 Follow-ups interviewers may request

- How to do ETA calculation (routing + historical traffic; precompute tiles)
- How surge pricing works (real-time demand/supply signals; ML)
- How to scale cross-region and cross-city
- How to ensure system is fair to drivers (assignment policies)

Presentation tips (how to speak these in an interview)

1. **Start with requirements** (explicit: functional vs non-functional). Ask clarifying Qs if the interviewer prompts.
2. **State high-level approach** in one sentence (e.g., “stateless API + cache + durable order DB; inventory reserved via atomic reservation service”). Draw a small diagram.
3. **Walk through one read and one write flow** (e.g., product page read & checkout write).
4. **Discuss bottlenecks & trade-offs** (e.g., push vs pull, strong vs eventual consistency).
5. **Give a short capacity estimate** to show quantitative thinking — state your assumptions and show the arithmetic.
6. **Finish with ops & monitoring** and list follow-up features you’d add if time permits.

Quick checklist: what an interviewer expects you to mention

- Clear requirements & trade-offs
- Data partitioning & scaling strategy
- Caching & CDN where appropriate
- Asynchronous & message-based components for heavy work
- Consistency model and how you maintain correctness (e.g., inventory reservation, driver assignment)
- Failure modes & recovery: leader elections, retries, DLQs, reconciliation
- Metrics + alerts + SLAs, and security basics
- Rough capacity & cost analysis (assumptions + calculations)